

Informe - Laboratorio Nro 1: Introducción a Spring Boot

Arquitectura de Software · Aplicación bancaria (backend Spring Boot + frontend React)

1. Objetivo

Simular las transacciones de un banco: crear y consultar clientes, transferir dinero entre cuentas y consultar el histórico de transacciones. El ejercicio de entrega obligatoria consiste en desarrollar un frontend básico que consuma los endpoints del backend, con mínimo tres vistas.

2. Revisión del backend (diegobotia/lab12026p)

El backend usa Spring Boot 3.5, Java 17, Spring Data JPA, MySQL, Lombok y MapStruct, organizado por capas (controller, service, repository, entity, dto, mapper). Sigue casi al pie de la letra la guía del laboratorio.

Método	Ruta	Función
GET	/api/customers	Listar clientes
GET	/api/customers/{id}	Consultar un cliente
POST	/api/customers	Crear cliente
POST	/api/transactions	Transferir dinero entre cuentas
GET	/api/transactions/{accountNumber}	Histórico de una cuenta (enviadas y recibidas)

Observaciones sobre el backend (no se modificó, por indicación del usuario):

- No configura CORS; por eso el frontend usa un proxy de Vite hacia el puerto 8080.
- *TransferMoney* no es transaccional (`@Transactional`) y no valida montos nulos, cero o negativos.
- *getCustomerById* lanza `RuntimeException` (HTTP 500) en lugar de 404.
- `TransactionMapper` no usa `componentModel = "spring"`; el constructor con parámetros de `CustomerDTO` está vacío.
- La guía menciona GET `/api/transactions`, GET por ID, PUT y DELETE que el repositorio no implementa.
- Credenciales root/root en `application.properties`.

3. Frontend desarrollado

Aplicación React (Vite) con Axios, en la carpeta **frontend**. Tres vistas, navegables mediante pestañas:

- **Clientes:** tabla con todos los clientes y formulario para crear uno nuevo.
- **Transferir:** selección de cuenta origen y destino (con saldo visible) y monto; muestra confirmación o el mensaje de error del backend.
- **Histórico:** al elegir un cliente se muestra la tabla de sus transacciones, indicando si fueron enviadas o recibidas, con monto y fecha.

Archivo	Descripción
src/api/bancoApi.js	Cliente Axios y funciones para cada endpoint
src/views/Customers.jsx	Vista de consulta y creación de clientes
src/views/Transfer.jsx	Vista de transferencia
src/views/History.jsx	Vista del histórico por cliente
vite.config.js	Proxy /api -> http://localhost:8080 (evita CORS)

4. Pruebas realizadas

Se levantó el backend con MySQL local y el frontend en modo desarrollo. Mediante el proxy (<http://localhost:5173/api>) se verificó:

- Creación de dos clientes (cuentas 111 y 222): respuesta 200 con los datos creados.
- Transferencia de 150 de la cuenta 111 a la 222: transacción registrada con ID y fecha.
- Histórico de la cuenta 222: devuelve la transacción recibida.
- *npm run build* compila sin errores.

Las vistas no se probaron manualmente en un navegador; la verificación se hizo sobre la API a través del proxy y la compilación del proyecto.

5. Cómo ejecutar

1. Backend (carpeta backend), con MySQL en el puerto 3306 y la base lab12026p creada:

```
./mvnw spring-boot:run
```

2. Frontend (carpeta frontend):

```
npm install
```

```
npm run dev
```

3. Abrir <http://localhost:5173>.

6. Entrega

Repositorio: <https://github.com/DavidAraangoPineda/lab1-banco> (privado, backend y frontend). Fecha límite indicada en el enunciado: 15 de septiembre de 2026. Si el repositorio debe ser visible para el profesor, hay que hacerlo público o darle acceso.